# DrawLang — Interpreter and LLVM Compiler

A painter-style language that outputs SVG, built with Flex, Bison, C and LLVM

## 1. Overview

DrawLang is a small language for drawing pictures with code. A DrawLang program ( `.draw` file) describes shapes, colours and positions, and produces an SVG image called **`picture.svg`** that any web browser can open.

The project has two phases. Both share one lexer, one set of compile-time checks and one SVG runtime, so they always give the same picture.

| Phase | Program | Pipeline |
|---|---|---|
| 1 – Interpreter | `drawi` | `source.draw → Flex → Bison (builds tree, checks, executes) → picture.svg` |
| 2 – AOT compiler | `drawc` | `source.draw → Flex → Bison (builds tree, checks, prints LLVM IR) → program.ll → llc → program.o → clang + runtime.o → executable → picture.svg` |

Everything runs on Linux. Section 2 installs the tools, section 3 builds the project, section 4 runs it and section 6 shows what each example must produce.

## 2. Installing the dependencies on Linux

You need: **flex**, **bison**, **gcc**, **make**, **llvm** (gives `llc`) and **clang**. The project was tested with flex 2.6.4, bison 3.8.2, gcc 13, LLVM/clang 18 on Ubuntu 24.04. Any LLVM version 15 or newer works (the generated IR uses opaque `ptr` pointers).

**Ubuntu / Debian:**

```
sudo apt-get update
sudo apt-get install -y flex bison gcc make llvm clang
```

**Fedora / RHEL:**

```
sudo dnf install -y flex bison gcc make llvm clang
```

**Arch Linux:**

```
sudo pacman -S --needed flex bison gcc make llvm clang
```

Check that every tool is found:

```
flex --version      # flex 2.6.x
bison --version     # bison (GNU Bison) 3.x
gcc --version
llc --version       # LLVM version 15 or newer
clang --version
```

**If `llc` or `clang` is "not found"** but the packages are installed, your distribution installed them with a version suffix (for example `llc-18`). Either pass the names to `make`:

```
make LLC=llc-18 CLANG=clang-18 test
```

or add the LLVM directory to your `PATH`, for example `export PATH=/usr/lib/llvm-18/bin:$PATH`.

# 3. Building the project

From the project directory:

```
make            # builds both phases
```

This runs Bison on `src/interp.y` and `src/compiler.y`, runs Flex on `src/drawlang.l` (once for each parser), and compiles:

- `./drawi` – the Phase 1 interpreter,
- `./drawc` – the Phase 2 compiler,
- `build/runtime.o` – the runtime that compiled programs are linked with.

You can also build one phase only:

```
make interpreter    # only ./drawi
make compiler       # only ./drawc and build/runtime.o
make clean          # delete everything that was built
```

Bison prints no warnings. The grammar has one expected shift/reduce conflict, declared with `%expect 1` (explained in section 5.6).

# 4. Running

## 4.1 Phase 1 – the interpreter

```
./drawi                         # reads source.draw in the current directory
./drawi examples/02_rosette.draw
```

It prints `wrote picture.svg` and writes `picture.svg` to the current directory. Open that file in a web browser (for example `firefox picture.svg` or `xdg-open picture.svg`).

Using make: `make run-interp SRC=examples/02_rosette.draw` (default `SRC` is `source.draw`).

### 4.2 Phase 2 – the compiler

The complete pipeline is one command:

```
        make run-compiled SRC=examples/02_rosette.draw
```

It runs these steps, which you can also type by hand:

```
        ./drawc examples/02_rosette.draw -o program.ll    # DrawLang -> LLVM IR (text)
        llc -filetype=obj -relocation-model=pic program.ll -o program.o   # IR -> object
file
        clang program.o build/runtime.o -lm -o program    # link with the SVG runtime
        ./program                                         # writes picture.svg
```

`drawc` usage: `./drawc [file.draw] [-o output.ll]`. The defaults are `source.draw` and `out.ll`. `program.ll` is ordinary readable LLVM IR. You can open it in a text editor. `make run-compiled` puts its files in `build/out/`.

### 4.3 Testing both phases together

```
        make test
```

`test.sh` runs every program in `examples/` through both phases. It checks that:

- the two `picture.svg` files are **byte-for-byte identical**,
- the printed messages and exit codes are identical,
- the `*error*` examples are rejected and the others succeed.

Expected output:

```
PASS  examples/01_sample.draw  (identical SVG, 24 lines)
PASS  examples/02_rosette.draw  (identical SVG, 55 lines)
PASS  examples/03_recursion.draw  (identical SVG, 22 lines)
PASS  examples/04_town.draw  (identical SVG, 44 lines)
PASS  examples/05_error_drawing.draw  (both phases rejected it with the same errors)
PASS  examples/06_error_names.draw  (both phases rejected it with the same errors)
PASS  examples/07_error_runtime.draw  (both phases rejected it with the same errors)
7 passed, 0 failed
```

The output of each run is kept in `build/test/<example>/interp/` and `build/test/<example>/comp/`.

## 5. The DrawLang language

### 5.1 Painter’s vocabulary

| Concept | DrawLang | Example |
|---|---|---|
| variable | `ink` | `ink i = 0;` then `i = i + 1;` |
| if / else | `when` / `otherwise` | `when (x > 1) { ... } otherwise { ... }` |
| else-if | `otherwise when` | `... otherwise when (x > 0) { ... } otherwise { ... }` |
| while loop | `swirl` | `swirl (i < 3) { ... }` |
| counted loop | `stamp N times` | `stamp 5 times { ... }` |
| function | `pattern` | `pattern flower(x, y, size) { ... }` |
| return | `give` | `give n * 2;` |

## 5.2 Drawing statements

| Statement | Meaning |
|---|---|
| `canvas W H;` | Image size. Must be the **first** statement, with positive number literals. |
| `pen "colour";` | Colour of all later shapes (default `black`). Accepts any of the 147 SVG colour names (any letter case) or `#rgb` / `#rrggbb`. |
| `circle(x, y, r);` | Filled circle with centre (x, y) and radius r. |
| `rect(x, y, w, h);` | Filled rectangle; (x, y) is the top-left corner. |
| `line(x1, y1, x2, y2);` | Line, 2 pixels wide. |
| `triangle(x1, y1, x2, y2, x3, y3);` | Filled triangle. |
| `text(x, y, size, "words");` | Text whose baseline starts at (x, y); `size` is the font size. |
| `slide dx dy;` | Moves the drawing position. All later shapes are drawn shifted by the total of all slides so far (also inside patterns). It never resets by itself: slide back with negative values. |
| `spin(angle) { ... }` | Rotates every shape in the block by `angle` degrees (clockwise), around the current drawing position. |
| `grow(factor) { ... }` | Scales every shape in the block by `factor` (must be > 0), around the current drawing position. |

Shape arguments, slide amounts, angles and factors may be any expressions. `spin` and `grow` blocks can be nested.

## 5.3 Expressions

- Numbers are double precision (`3`, `2.5`).
- Arithmetic: `+ - * / %` (`%` is floating-point remainder) and unary `-`.
- Comparison: `< > <= >= == !=` give `1` (true) or `0` (false).

- Logic: `&&`, `||` (short-circuit) and `!`. Any non-zero value is true.
- Variables, brackets, and pattern calls such as `fib(n - 1)`.
- Precedence, from low to high: `||`, `&&`, `== !=`, `< > <= >=`, `+ -`, `* / %`, unary.

## 5.4 Variables and patterns

- `ink name = expr;` declares a variable; `name = expr;` changes it. A variable must be declared (textually) before it is used.
- The main program and each pattern have their own variables. A pattern sees only its parameters and the `ink`s declared inside it. Patterns cannot see the main program's variables; pass values in as arguments instead.
- Patterns are defined at the top level and can be called before their definition, from other patterns, and recursively.
- A call can be a statement (`flower(80, 100, 20);`) or part of an expression (`ink h = fib(i) * 5;`). A pattern that ends without `give` gives `0`.
- `stamp N times` evaluates `N` once and runs the block while a counter (0, 1, 2, ...) is less than `N`. So `stamp 2.5 times` runs 3 times and a negative `N` runs 0 times.
- Comments: `// to end of line` and `/* ... */`.

## 5.5 Compile-time checks

Before anything runs (Phase 1) or any IR is written (Phase 2), the checker reads the whole program and reports **every** error it finds, each with its line number. Then it stops with exit code 1, and no `picture.svg` is written.

| **Check** | **Example message** |
|---|---|
| Shape outside the canvas | `file:6: error: rect(150, 150, 100, 20) is outside the 200x200 canvas` |
| Negative size or radius | `file:5: error: circle radius is negative (-5)` |
| Unknown colour name | `file:4: error: unknown colour "pinkish"` |
| Wrong argument count (shape) | `file:8: error: circle expects 3 argument(s) but got 2` |
| Wrong argument count (pattern) | `file:10: error: pattern 'dot' expects 2 argument(s) but got 3` |
| Undeclared variable | `file:11: error: undeclared variable 'b'` |
| Others | unknown pattern, variable declared twice, `give` outside a pattern, missing/misplaced `canvas`, `grow` factor not positive, syntax errors, unexpected characters |

**Values only known while running.** The canvas and size checks need numbers. At compile time the checker computes every shape argument that is a constant (numbers, and arithmetic on numbers) and follows constant `slide`s in the main program. Those shapes are fully checked before the program runs. A shape whose position depends on a variable, a loop or a pattern parameter (like `circle(x, 50, 10)` in a loop) cannot be checked before it runs. The runtime

checks it with **exactly the same rules** at the moment it is drawn, and stops with its line number, for example:

```
line 7: runtime error: circle(200, 50, 10) is outside the 200x100 canvas
```

The partial `picture.svg` is then deleted. Both phases behave identically here, because both use the same runtime check (`dl_shape_error` in `runtime.c`).

The canvas check uses the shape's position after `slide`s. It is done in the shape's own unrotated, unscaled frame: `spin` and `grow` are not taken into account.

### 5.6 One grammar note

`slide` takes two expressions with no comma, so `slide 20 -5;` means dx = 20, dy = −5. The only ambiguity is `slide a (b);`, which could mean `slide a b` or a slide by a single value `a(b)` (a pattern call). Bison resolves this as the pattern call, like every identifier followed by `(`. Write `slide a b;` instead. This is the single shift/reduce conflict, declared with `%expect 1`.

## 6. Examples and expected output

All examples are in `examples/`. `source.draw` (the default input) is the same program as `examples/01_sample.draw`.

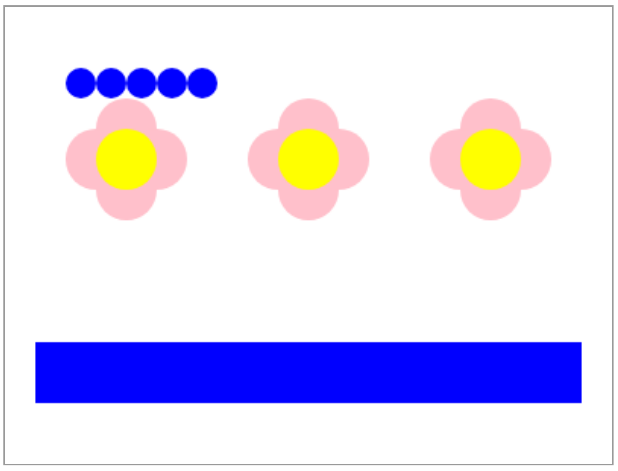
### 6.1 `01_sample.draw` – the sample from the proposal

```
canvas 400 300;
pattern flower(x, y, size) {
  pen "pink";
  circle(x, y - size, size); circle(x, y + size, size);
  circle(x - size, y, size); circle(x + size, y, size);
  pen "yellow";
  circle(x, y, size);
}
ink i = 0;
swirl (i < 3) { flower(80 + i * 120, 100, 20); i = i + 1; }
ink level = 2;
when (level > 1) {
  when (level > 5) { pen "red"; } otherwise { pen "blue"; }
  rect(20, 220, 360, 40);
}
stamp 5 times { circle(50, 50, 10); slide 20 0; }
```

```
        $ ./drawi examples/01_sample.draw
        wrote picture.svg
        $ make run-compiled SRC=examples/01_sample.draw      # same picture.svg
```

Expected: three pink flowers with yellow centres, a blue bar, and a row of five blue circles at x = 50, 70, 90, 110, 130. The circles are blue because the pen is still blue from the bar. Start of `picture.svg`:

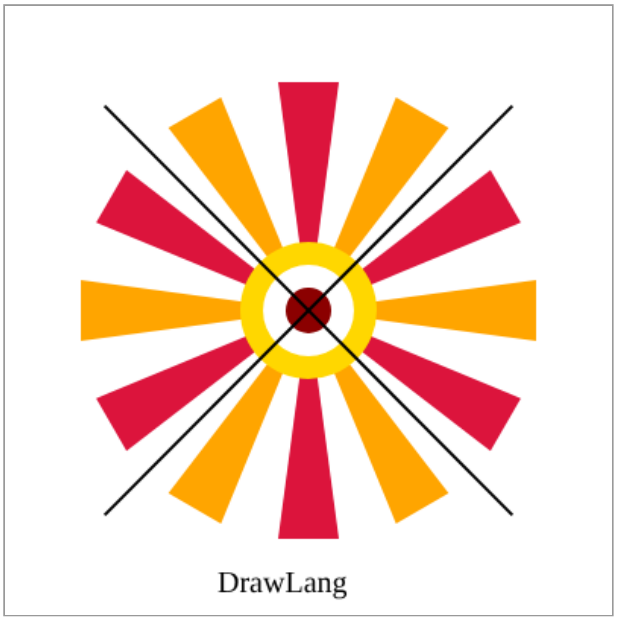
```
        <svg xmlns="http://www.w3.org/2000/svg" width="400" height="300" viewBox="0 0 400
300">
          <rect width="400" height="300" fill="white"/>
          <circle cx="80" cy="80" r="20" fill="pink"/>
          <circle cx="80" cy="120" r="20" fill="pink"/>
          ...
          <rect x="20" y="220" width="360" height="40" fill="blue"/>
          <circle cx="50" cy="50" r="10" fill="blue"/>
          <circle cx="70" cy="50" r="10" fill="blue"/>
          ...
        </svg>
```



01_sample.draw

## 6.2 `02_rosette.draw` – spin, grow, triangle, line, text, otherwise when

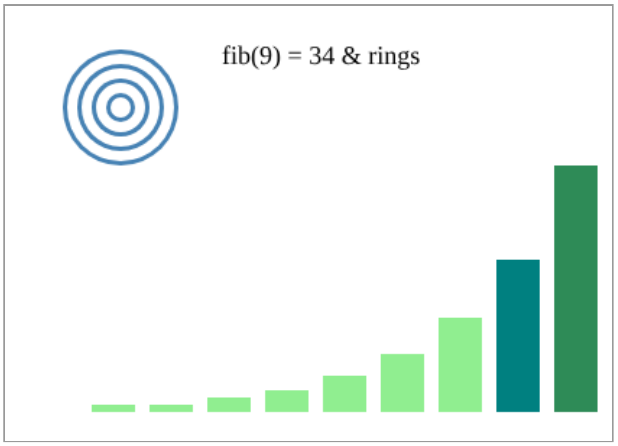
It slides to the centre of a 400×400 canvas and draws 12 triangles. Each one is rotated by a further 30° with `spin`. Three rings are drawn by scaling one circle with `grow`. Two rotated lines and a text label complete the picture.




02_rosette.draw

## 6.3 `03_recursion.draw` – patterns with give, recursion, && and ||

`fib(n)` is a recursive pattern that gives values. It sets the heights of a bar chart of the first ten Fibonacci numbers, with colours chosen by `&&` / `||` conditions. `rings` is a recursive pattern used as a statement.




03_recursion.draw

## 6.4 `04_town.draw` – slide and patterns

A `house` pattern is stamped five times. Each `slide 95 0` moves the next house to the right. The program then slides back and draws a dashed road with `swirl`.



04_town.draw

## 6.5 `05_error_drawing.draw` – drawing errors (compile time)

```
        $ ./drawi examples/05_error_drawing.draw          # or ./drawc
examples/05_error_drawing.draw
        examples/05_error_drawing.draw:4: error: unknown colour "pinkish"
        examples/05_error_drawing.draw:5: error: circle radius is negative (-5)
        examples/05_error_drawing.draw:6: error: rect(150, 150, 100, 20) is outside the
200x200 canvas
        examples/05_error_drawing.draw:7: error: line(10, 10, 10, 250) is outside the
200x200 canvas
        examples/05_error_drawing.draw:8: error: grow factor must be positive (-2)
        examples/05_error_drawing.draw:9: error: text size is negative (-3)
```

Exit code 1. No picture and (for `drawc`) no `.ll` file is produced.

## 6.6 `06_error_names.draw` – name and argument errors (compile time)

```
        $ ./drawi examples/06_error_names.draw           # or ./drawc ...
        examples/06_error_names.draw:8: error: circle expects 3 argument(s) but got 2
        examples/06_error_names.draw:9: error: triangle expects 6 argument(s) but got 4
        examples/06_error_names.draw:10: error: pattern 'dot' expects 2 argument(s) but
got 3
        examples/06_error_names.draw:11: error: undeclared variable 'b'
        examples/06_error_names.draw:12: error: undeclared variable 'size'
        examples/06_error_names.draw:13: error: unknown pattern 'star'
```

### 6.7 `07_error_runtime.draw` – a shape that leaves the canvas while running

The circle's x grows inside a loop, so it cannot be checked at compile time (section 5.5). Both phases stop at the same circle:

```
        $ ./drawi examples/07_error_runtime.draw
        line 7: runtime error: circle(200, 50, 10) is outside the 200x100 canvas
        $ make run-compiled SRC=examples/07_error_runtime.draw
        ...
        line 7: runtime error: circle(200, 50, 10) is outside the 200x100 canvas
```

## 7. How it works

**Lexer (`drawlang.l`).** Turns the text into tokens: keywords, the five shape names (one `SHAPE` token that carries the shape kind), numbers, strings, identifiers and operators. Each token records its line number.

**Parser actions build a tree.** In both `.y` files, the Bison actions build a syntax tree (`ast.h`). The tree mirrors the source one-to-one: a `swirl` node holds its condition and body. Loops and pattern calls need to run a piece of code many times, and the tree lets them do that. It is not an IR, bytecode or any other intermediate code.

**Checks (`check.c`).** The action of the start rule `program` runs once the whole file is parsed. It calls `check_program`, which reports all compile-time errors. It also links each variable use to a numbered *slot* in its pattern's (or the main program's) frame, and each call to its pattern.

**Phase 1 – execute (`interp.y`).** After the checks, the same action walks the tree and executes it directly. Variables live in an array of doubles. Each pattern call gets a fresh array. `give` sets a flag that stops the enclosing loops and blocks. Drawing statements call the runtime (`svg_circle`, ...).

**Phase 2 – emit LLVM IR (`compiler.y`).** After the checks, the same action walks the tree and **prints LLVM IR instead of executing**:

- each pattern becomes `define double @dl_<name>(double ...)`, and the main program becomes `define i32 @main()`;
- each variable slot becomes an `alloca double`, read with `load` and written with `store`;
- arithmetic becomes `fadd/fsub/fmul/fdiv/frem/fneg`, comparisons become `fcmp` + `uitofp`, and `&&`/`||` become branches joined by a `phi`;
- `when`, `swirl` and `stamp` become basic blocks joined by `br`;
- `give` becomes `ret`. It first closes any `spin`/`grow` group still open in that pattern.

- Drawing statements become calls such as `call void @svg_circle(i32 4, double %t0, double %t3, double %t4)`, where the first argument is the source line for run-time errors;
- colour names and text labels become global string constants;
- number constants are written in LLVM's exact hexadecimal form, so no precision is lost.

**Runtime (`runtime.c`).** These are small C functions that print SVG text: `svg_begin`, `svg_pen`, `svg_slide`, `svg_circle`, `svg_rect`, `svg_line`, `svg_triangle`, `svg_text`, `svg_spin_begin`, `svg_grow_begin`, `svg_group_end` and `svg_end`. A transform block becomes an SVG group `<g transform="rotate(a x y)">` or `<g transform="translate(..) scale(f) translate(..)">` around its shapes. The interpreter calls these functions directly, and the compiled program calls them from LLVM IR. Because both use the same functions, the output is identical.

# 8. Project files

| File | What it is |
|---|---|
| `README.pdf` | This document. |
| `README.md` | Source of this document (Markdown). |
| `Makefile` | Builds both phases; targets `all`, `interpreter`, `compiler`, `run-interp`, `run-compiled`, `test`, `clean`. |
| `test.sh` | Runs all examples through both phases and compares the results (`make test`). |
| `source.draw` | Default input program (the sample from the proposal). |
| `src/drawlang.l` | **Flex** lexer, shared by both phases (compiled once for each parser). |
| `src/interp.y` | **Bison** grammar for Phase 1. Its actions build the tree, run the checks and execute the program (tree-walking evaluator at the end of the file). Produces `drawi`. |
| `src/compiler.y` | **Bison** grammar for Phase 2. The same grammar; its actions build the tree, run the checks and print LLVM IR (code generator at the end of the file). Produces `drawc`. |
| `src/ast.h`, `src/ast.c` | The syntax-tree node type and the helpers the Bison actions use to build it. |
| `src/check.c` | Compile-time checks shared by both phases (colours, argument counts, variables, canvas bounds, sizes), plus variable-slot resolution. |
| `src/runtime.c`, `src/runtime.h` | **Runtime C file**: the `svg_*` helpers that write `picture.svg`, and the shape rules (`dl_shape_error`) used by both the compile-time and run-time checks. |
| `examples/01_sample.draw` | The required sample program. |
| `examples/02_rosette.draw` | spin, grow, triangle, line, text, `otherwise when`. |
| `examples/03_recursion.draw` | Patterns with `give`, recursion, `&&`, `\|\|`, `%`. |
| `examples/04_town.draw` | slide, patterns, `stamp`, `swirl`, hex colours. |
| `examples/05_error_drawing.draw` | Compile-time errors: outside canvas, negative sizes, unknown colour. |
| `examples/06_error_names.draw` | Compile-time errors: argument counts, undeclared variables, unknown pattern. |
| `examples/07_error_runtime.draw` | Run-time canvas check for a shape whose position is computed in a loop. |
| `docs/images/*.png` | Pictures of the expected output, used in this document. |

Files created by building (deleted by `make clean`): `drawi`, `drawc`, `build/` (generated parsers and lexers, `runtime.o`, test results, `build/out/program*`), and `picture.svg`.